

Set Explaining for LCPC

What is set?

`set` is a container in C++ STL that stores **unique elements** in a sorted order.

Unlike `map`, `set` stores only values, not key-value pairs.

General form:

```
set<data_type> name;
```

Example:

```
set<int> s;
```

This creates a set that stores integers.

Adding elements

We use:

```
s.insert(value);
```

Example:

```
s.insert(5);  
s.insert(10);  
s.insert(5);
```

The result:

```
5  
10
```

The second `5` is ignored because `set` does not allow duplicates.

Important properties of set

1. Unique elements

Example:

```
set<int> s;  
  
s.insert(1);  
s.insert(1);  
s.insert(2);
```

The set contains:

```
1  
2
```

Only one copy of each element exists.

2. Sorted automatically

If we insert:

```
s.insert(10);  
s.insert(3);  
s.insert(7);
```

The set stores:

```
3  
7  
10
```

Elements are always sorted in ascending order.

Checking if an element exists

Use:

```
s.count(value)
```

Example:

```
if(s.count(5))
{
    cout << "Found";
}
else
{
    cout << "Not Found";
}
```

`count()` returns:

- `1` if the element exists.
- `0` if it does not exist.

Iterating through a set

We can print all elements using:

```
for(auto x : s)
{
    cout << x << endl;
}
```

Example:

Set:

```
3
5
8
```

Output:

```
3  
5  
8
```

Removing elements

Use:

```
s.erase(value);
```

Example:

```
s.erase(5);
```

The element `5` will be removed.

Size of set

Use:

```
s.size();
```

Example:

```
cout << s.size();
```

If the set contains:

```
1  
2  
3
```

The output is:

3

Clearing the set

Remove all elements:

```
s.clear();
```

After:

```
s.clear();
```

The set becomes empty.

Finding elements

Another way to check existence:

```
auto it = s.find(value);

if(it != s.end())
{
    cout << "Found";
}
```

If the element exists, `find()` returns an iterator pointing to it.

Otherwise, it returns:

```
s.end()
```

Common Competitive Programming Uses

1. Removing duplicates

Example:

Array:

```
5 7 5 8 7
```

Using set:

```
set<int> s;  
  
for(int x : arr)  
{  
    s.insert(x);  
}
```

Result:

```
5  
7  
8
```

2. Counting distinct elements

Instead of manually checking duplicates:

```
cout << s.size();
```

Example:

```
1 2 2 3 3 3
```

The set contains:

```
1  
2  
3
```

So:

```
s.size() = 3
```

3. Checking if something appeared before

Example:

```
if(s.count(x))  
{  
    cout << "Already exists";  
}  
else  
{  
    s.insert(x);  
}
```

Useful for:

- Detecting duplicates.
- Tracking visited elements.

Time Complexity

For `set`:

- Insert: $O(\log n)$
- Search: $O(\log n)$
- Delete: $O(\log n)$

Because `set` is usually implemented using a balanced tree.

Difference between map and set

map	set
Stores key and value	Stores only values
key -> value	value
Used for counting and mapping	Used for unique elements
Example: name -> age	Example: unique numbers

Example Problems

- Codeforces 236A - Boy or Girl
- Codeforces 228A - Is your horseshoe on the other hoof?
- Codeforces 443A - Anton and Letters

Summary

Think about `set` as a bag that:

- Never stores duplicates.
- Keeps elements sorted.
- Allows fast searching.

The most important operations:

```
s.insert(x);
s.erase(x);
s.count(x);
s.size();

for(auto x : s)
{
    cout << x;
}
```

`set` is one of the most useful STL containers for competitive programming, especially for problems involving distinct elements and duplicate detection.